Custom Programming Language Interpreter in Java

Demo Guide and Project Explanation

1. Project Overview

This project is a custom programming language interpreter built completely from scratch in Java. The purpose is to understand how programming languages work internally, from reading source code to executing output.

The project does not use parser generators such as ANTLR or JavaCC. Every major stage is manually implemented using Java Standard Edition.

2. Main Pipeline

The interpreter follows this complete flow:

```
Source Code
-> Lexer
-> Tokens
-> Parser
-> AST
-> Interpreter
-> Output
```

This is called a tree-walking interpreter because the interpreter walks through the Abstract Syntax Tree and executes each node.

3. Supported Language Features

- Variable declarations using let
- Variable assignment
- Print statements
- If / else conditions
- While loops
- For loops
- Block statements with braces
- Nested variable scopes
- Numbers, strings, booleans, and null
- Arithmetic operations: +, -, *, /
- Comparison operations: >, >=, <, <=
- Equality operations: ==, !=
- Unary operations: ! and -
- Line-number based lexer, parser, and runtime errors
- Interactive terminal REPL and script-file execution

4. Team Work Division

- Person 1: Architecture, AST classes, and Environment
- Person 2: Lexer and token generation
- Person 3: Parser and syntax analysis
- Person 4: Interpreter and execution

Each part connects to the next part through strict method signatures so the team can work independently and still integrate cleanly.

5. Main.java

Main.java is the entry point of the project. It decides whether to run a file or start the interactive prompt.

```
javac *.java
java Main
java Main test.script
```

The core pipeline in Main.java connects all team parts:

```
Lexer lexer = new Lexer(source);
List<Token> tokens = lexer.scanTokens();

Parser parser = new Parser(tokens);
List<Stmt> statements = parser.parse();

interpreter.interpret(statements);
```

6. Lexer

The lexer converts raw source code into tokens. Tokens are small structured pieces of the program, such as keywords, identifiers, numbers, strings, operators, and punctuation.

Example source code:

```
let x = 10;
```

Example token sequence:

```
LET IDENTIFIER EQUAL NUMBER SEMICOLON EOF
```

The lexer also tracks line numbers so errors can point to the exact line where a problem occurred.

7. Token and TokenType

TokenType.java defines every possible token category, such as LET, PRINT, IDENTIFIER, NUMBER, STRING, PLUS, MINUS, and EOF.

Token.java stores the actual token data:

```
TokenType type;
String lexeme;
Object literal;
int line;
```

For example, the number 10 can become a token like:

```
Token[NUMBER "10" 10.0 at line 1]
```

8. Parser

The parser receives the token list and checks whether the tokens follow the grammar of the language. It then builds an AST, which stands for Abstract Syntax Tree.

The required parser API is:

```
public Parser(List<Token> tokens)
```

```
public List<Stmt> parse()
```

Example valid code:

```
let x = 10;
```

Example invalid code:

```
let = 10;
```

The invalid code produces a parser error similar to:

```
[Line 1] Error at '=': Expected variable name.
```

9. Operator Precedence

The parser understands operator precedence. This means multiplication and division happen before addition and subtraction.

```
let x = 2 + 3 * 4;
print x;
```

The parser reads this as:

```
2 + (3 * 4)
```

So the output is:

```
14
```

10. For Loop Handling

For loops are supported safely by converting them into existing block and while-loop AST nodes. This avoids needing a separate ForStmt class.

Source code:

```
for (let i = 0; i < 3; i = i + 1) {
    print i;
}
```

Internal equivalent:

```
{
    let i = 0;
    while (i < 3) {
        print i;
        i = i + 1;
    }
}
```

11. AST

AST means Abstract Syntax Tree. It represents the structure of the program using Java objects instead of raw text.

Expressions are represented by Expr nodes such as BinaryExpr, LiteralExpr, VariableExpr, AssignExpr, UnaryExpr, and GroupingExpr.

Statements are represented by Stmt nodes such as PrintStmt, VarDeclareStmt, IfStmt, WhileStmt, BlockStmt, and ExpressionStmt.

Example code:

```
print 2 + 3;
```

Conceptual AST:

```
PrintStmt
  BinaryExpr
    left: 2
    operator: +
    right: 3
```

12. Visitor Pattern

The project uses the Visitor Design Pattern. Each AST node has an accept method, and the interpreter implements visitor methods for each node type.

```
visitBinaryExpr()
visitPrintStmt()
visitWhileStmt()
```

This keeps AST structure and execution logic cleanly separated.

13. Environment and Memory

Environment.java manages variables and scoped memory. It stores variable names and values in a map.

```
Map<String, Object> values
```

It also supports nested scopes using an enclosing Environment reference.

Example:

```
let value = 100;
{
    let value = 200;
    print value;
}
print value;
```

Output:

```
200
100
```

This shows scope isolation: the inner value does not destroy the outer value.

14. Interpreter

The interpreter receives List<Stmt> from the parser and executes each statement. It evaluates expressions, creates variables, updates variables, executes loops, checks conditions, and prints output.

Example:

```
print 10 + 20;
```

The interpreter evaluates the BinaryExpr and prints:

```
30
```

15. Complete Execution Example

Input program:

```
let x = 2 + 3 * 4;
print x;
```

Step 1: Source code is read.

Step 2: The lexer creates tokens.

```
LET IDENTIFIER EQUAL NUMBER PLUS NUMBER STAR NUMBER SEMICOLON PRINT
IDENTIFIER SEMICOLON EOF
```

Step 3: The parser creates an AST.

```
VarDeclareStmt
  name: x
  initializer: 2 + (3 * 4)

PrintStmt
  expression: VariableExpr x
```

Step 4: The interpreter executes the AST.

```
3 * 4 = 12
2 + 12 = 14
x = 14
print x
```

Final output:

```
14
```

16. Demo Commands

Open PowerShell in the project folder and run:

```
cd "C:\Users\ankit\OneDrive\Documents\Ashith
Anandnath\python\Interpreter\Interpreter"
javac *.java
java Main
```

17. Demo Code To Paste

Paste these lines into the terminal after java Main starts:

```
print "===== DEMO START =====";
let x = 2 + 3 * 4;
print x;
if (x > 10) { print "x is greater than 10"; } else { print "x is small"; }
let total = 0;
for (let i = 1; i <= 5; i = i + 1) { total = total + i; }
print total;
let value = 100;
{ let value = 200; print value; }
print value;
print true;
print null;
print "===== DEMO COMPLETE =====";
```

Expected important output:

```
===== DEMO START =====
14
x is greater than 10
15
200
100
true
nil
===== DEMO COMPLETE =====
```

18. What To Say During Demo

Our project is a custom interpreted language built from scratch in Java. The user writes code in our language. First, the lexer breaks the code into tokens. Then the parser checks the grammar and builds an AST. The AST represents the program structure. Finally, the interpreter walks the AST using the Visitor pattern and executes each statement. Variables are stored in an Environment class, which supports nested scopes. This lets us handle local and global variables properly.

The project does not rely on parser generators or external libraries. Everything is implemented manually using Java Standard Edition.

19. Strengths

- Built fully in Java
- No external parser generator
- Manual lexer
- Manual recursive-descent parser
- AST-based execution
- Visitor design pattern
- Nested environment for memory scope
- Custom error handling
- Line-number based errors
- Interactive REPL support
- Script file support
- For loops implemented through while-loop conversion

20. Limitations

- No functions yet
- No arrays yet
- No classes yet
- No input function yet
- No break or continue yet
- No advanced standard library yet

These features can be added later because the project architecture is modular. We can extend the lexer, parser, AST, and interpreter step by step.

21. Closing Line

This project helped us understand how programming languages work internally, from source code input to tokenization, parsing, AST construction, memory handling, and final execution.